2번 문제 설명 및 코드 설명

- **문제 2**

2-1. P2.1 스위치 누르면 ADC값을 0~2(소수점 첫째자리까지)로 매핑 및 세그먼트 출력

2-2. P2.1 스위치 누르면 출력된 시간 간격으로 P1.0과 P4.7 led가 번갈아가며 토글

2-3. P2.1 스위치 누르면 led 소등 후 *을 누르는 동안은 반시계 방향, #을 누르는 동안은 시계방향으로 모터 가속
(추가 점수1)
0에서 최대 속도로 도달하는 데에는 7초
(추가 점수2)
반대방향 버튼 push시 방향이 바로 바뀌는 것이 아니라 감속을 하다가 정지 후 반대방향으로 회전
(추가 점수3)
.pwm 30% 구간은 모터가 회전하지 않으므로 해당 구간 스킵

2-4 P2.1 스위치 누르면 2-3에서 설정한 속도 고정 & 초음파 센서로 측정되는 거리 세그먼트에 출력 및 10cm 이내로 물체 감지시 모터 정지

2-5. P2.1 스위치 누르면 2-3에서 고정한 속도로 모터 동작 & 손가락 혹은 물체에 걸려 모터 회전 방해시 모터 동작 정지

3-6. P2.1 스위치 누르면 초기 상태 복귀

코드 설명

- 코드 구조

문제 2.

```
Main while loop
switch(status)
    status = 0
    초기모드, 변수들을 초기화.
    또한 모터 작동시 엔코더 인터럽트가
    너무 많이 발생하므로 엔코더 인터럽트
    비활성화.

    status = 1
      ADC 값을 cycles 변수에 저장.
      0~2.0 매핑을 위해
       ADC값 변조.
        세그먼트에 cycles 값의 천,백의
        자리 및 소수점 표시.

      status = 2
        LED flag를 이용한 LED 토글

      status = 3
        LED off
         keypad 다이내믹 구동.
          이때 *, # 값만 이용.
             *이 눌렸을 때,
              star_pressed = 1
              모터가 반시계 방향으로 돌 경우
                  acc_flag = 1
                 시계방향인 경우
                  deacc_flag = 1
           #이 눌렸을 때
              hash_pressed = 1
                 모터가 시계 방향으로 돌 경우
                deacc_flag = 1
                반시계방향인 경우
                  acc_flag = 1
            현재 모터값 임시 저장

    status = 4
       초음파 센서를 위해 TACCR0 값 조정
           초음파 flag = 0 이면 trig 핀 10us 간 ON
               이후 flag = 1
      거리값 distance < 10 이면 모터 정지
       distance > 10 이면 임시 저장해둔
       모터값 복원.

     status = 5
       세그먼트 off
      정지 동작을 위해 TACCR0값 크게 조정
       (모터가 빨리 도는 경우를 위해)
        엔코더 인터럽트 활성화
```

Timer A0 interrupt

timer_01 증가

status = 2 (LED 토글 logic)

주어진 cycle 동안 LED_flag 토글.

status = 3 (모터 가속 logic)

- *이 눌렸을 때
  - acc_flag=1 이면 TA2CCR2++
    (반시계 방향 가속)
    - pwm duty 30% skip.
  - deacc_flag=1 이면 TA2CCR1--
    (시계 방향 감속)
    - TA1CCR1 의 duty가 30%가 되면
      TACCR1 = 0
- #이 눌렸을 때
  - acc_flag=1 이면 TA2CCR1++
    (시계 방향 가속)
    - pwm duty 30% skip.
  - deacc_flag=1 이면 TA2CCR2--
    (반시계 방향 감속)
    - TA1CCR2 의 duty가 30%가 되면
      TACCR2=0

status = 4

timer_01을 통한 세그먼트에 초음파 거리
출력

status = 5

모터 정지 logic.

현재 엔코더 값과

10000ms 전 엔코더 값의 차가

2 이하이면 모터 정지.

Port 2 Interrupt.

P2.1이 눌리면.

타이머 채터링 방지.

Status (상태) 0부터 5까지 순환.

status =5 이면
모터에 임시값 복원.

Timer A1 Interrupt

초음파 카운트 증가.

초음파 flag 초기화.

타이머 채터링 감소
(채터링 방지 목적.)

port 1 Interrupt

초음파 센서 logic.

distance 계산.

엔코더 값 지속적 증가

변수 설명

```c
unsigned int number[10] = { 0xDB, 0x50, 0x1F, 0x5D, 0xD4,
0xCD, 0xCF, 0xD8, 0xDF, 0xDD };
```

- 세그먼트 표현 0~9 배열

```c
unsigned int status = 0; // initial condition
```

- 현재 상태(state)를 제어하는 변수

```c
unsigned int cycles = 0;
```

- ADC값으로부터 LED 토글 사이클 결정 변수

```c
unsigned int timer_gl = 0;
```

- 1ms임을 이용하여 시간을 계산하는 타이머 변수

```c
unsigned short LED_flag = 0;
```

- Led 토글에 사용할 led flag 변수

```c
unsigned long ultra_time = 0;
```

- 초음파 센서에 사용할 초음파가 되돌아오기까지 걸린 시간

```c
unsigned int distance = 9999; //data initialize 9999
```

- 초음파센서로부터 계산한 거리(cm)

```c
unsigned int ultra_flag = 0;
```

- 초음파센서 Trig 제어를 위한 flag 변수

```c
unsigned int temp_CCR1 = 0;
unsigned int temp_CCR2 = 0;
```

- 모터 값을 임시 저장하는 변수.

```c
int delta_enc = 0;
```

- 모터 정지 logic 에서 10000us 이전에 비해 달라진 엔코더 값

```c
int cycle_counter = 0;
```

- 모터 정지 logic 에서 사용할 엔코더 카운트 변수

```c
volatile unsigned short star_pressed = 0;
```

- 키패드 *이 눌렸는지 확인하는 변수

```c
volatile unsigned short hash_pressed = 0;
```

- 키패드 #가 눌렸는지 확인하는 변수

```c
volatile unsigned short acc_flag = 0;
```

- 모터를 가속할지 결정하는 flag 변수

```c
volatile unsigned short deacc_flag = 0;
```

- 모터를 감속할지 결정하는 flag 변수

```c
unsigned int timer_chattering = 0; // 50 * 1000
```

- Timer 을 이용한 채터링 방지 변수

문제를 해결한 주요 logic

1. ADC 값 0~2 매핑

1.1) ADC 의 값의 범위는 대략 1200~ 4095 이므로, 이를 1099 ~ 2099 (소숫점으로 표현할 시 1.0~2.0) 으로 매핑하기 위한 계산.

1.2) Raw ADC 값에 1100 을 뺀 후, 3 을 나눈다. 이후 2 bit 만큼 shift right (%4) 한다.

2. 매핑된 값의 시간만큼 LED 토글

2.1) 1ms 마다 타이머 인터럽트가 발동되도록 실행하여 timer_gl 을 사용한다.

1ms 마다 timer_gl++을 해 주면 timer_gl의 천의자리와 백의자리값을 이용하여 소숫점 한 자리의 초(sec)를 표현 할 수 있다.

2.2) 앞서 매핑한 값을 바탕으로 timer_gl 값이 cycle 값에 도달 할 때 마다 led flag 를 토글시켜준다.

3. 모터 가속/감속 logic

3.1) TA2CCR0 값을 10000 으로 설정하고, 한 사이클 감속/감속 을 1ms 마다 수행함으로써 0 부터 10000 까지 걸리는데 10 초가 되도록 설정하였다. (1ms * 10000 = 10sec)

3.2) TA2CCR1 값과 TA2CCR2 값을 비교하여 현재 모터가 시계방향인지, 반시계방향으로 회전하는지 파악하고, 어떤 버튼이 눌린 상태인지에 따라 가속/감속을 결정한다.

3.3) Pwm duty (3000) 구간을 skip 함으로써 결과적으로 가속/감속에 7 초가 걸리도록 설계하였다.

4. 엔코더 인터럽트 관련

4.1) 초음파 센서로 값을 받아올 때, 엔코더 인터럽트가 켜져있음으로써 너무 많은 인터럽트가 발생하여 원하는 동작을 하지 않았다. 따라서 필요에 따라, 즉 status = 5 인 엔코더를 사용하는 모터 정지 state 가 아닌 경우에 encoder interrupt 를 비활성화 해주었다.

5. 모터 정지 logic

5.1) 10000us 마다 인터럽트를 통해 현재 엔코더값과 과거의 엔코더값(10000us 전)을 비교함으로써 모터 회전에 방해가 되었는지 판단하였다. 매 사이클의 끝에서 cycle_counter 를 0 으로 초기화 시킨 후, 다음 사이클의 시작에서 현재 encoder 값과 비교하였다. 이때 손으로 모터를 멈추는 경우 오차가 발생할 수 있으므로 두 값이 같은지가 아닌 두 값의 차이가 2 이하인지 판단하도록 하였다.

5.2) 또한 모터가 시계방향으로 돌거나 반시계방향으로 돌 수 있으므로, 두 방향 모두 cycle_counter 값을 증가시키는 방향으로 설계하였다.

6. Timer 를 이용한 채터링 방지

6.1) 채터링에 timer 을 이용하였다. 한 번 인터럽트가 발생하면 timer_chattering 에 1000 의 값을 준다. 50us * 1000, 즉 50000us 가 되면 timer_chattring = 0 이 된다.

6.2) 이후 status 를 증가시키기 위해 버튼을 누를 때 timer_chatterint = 0 이어야만 동작하게 하여 채터링을 방지하였다.